Machine Learning for Finance

Project Report

Neural Network Based European Option Pricing

under Black-Scholes SDE Dynamics

A supervised learning study of neural networks as pricing surrogate models

Matteo Giorgi
Lorenzo Ghiotti
Nalumansi Anne Frances
Alice Pedron

May 30, 2026

Abstract

This project studies whether a feed-forward neural network can accurately approximate the Black-Scholes pricing function for European call options. The problem is addressed in a controlled setting: data are generated synthetically from the analytical Black-Scholes formula, while Monte Carlo simulation is used as a numerical benchmark. The neural network is trained in a supervised learning framework to learn the mapping from market and contract parameters to the theoretical option price. The goal is not to build a real trading system, but to evaluate the role of a neural network as a pricing surrogate in a mathematically well-defined financial setting.

Contents

1 Introduction

1.1 Motivation

Derivative pricing is one of the classical problems in quantitative finance. For European options, the Black-Scholes model provides a closed-form price under precise assumptions on the dynamics of the underlying asset, market completeness, and constant volatility [1]. The availability of an analytical solution makes Black-Scholes an ideal controlled environment for evaluating numerical methods and machine learning models.

At the same time, many relevant pricing problems do not admit simple closed-form solutions. In such cases, classical numerical methods such as Monte Carlo simulation are flexible but can become computationally expensive, especially when prices must be evaluated repeatedly across many contracts, parameter configurations, or risk scenarios [3]. This motivates the study of neural networks as pricing surrogate models. The idea is not to replace the mathematical pricing model itself, but to learn a fast approximation of its input-output map. After training, a neural network can produce prices through a single forward pass, which is typically much faster than running a new Monte Carlo simulation for each option.

1.2 Project Goal

The goal of this project is to study whether a feed-forward neural network can approximate the Black-Scholes pricing function for European call options. The project is not intended to build a trading system or to predict real market prices. Instead, it focuses on a mathematically controlled supervised learning problem where the target function is known exactly. This controlled setting is important because it allows the approximation error of the neural network to be measured directly against the analytical Black-Scholes price.

Table 1.1 summarizes the scope of the project.

Aspect	Included in the project	Not included in the project
Option type	European call options	American or exotic options
Data source	Synthetic Black-Scholes data	Exchange-traded market option data
Target value	Analytical Black-Scholes price	Observed market transaction price
Main model	Feed-forward neural network	Trading or forecasting system
Benchmarks	Black-Scholes, Monte Carlo, SVR	Production pricing engine
Main objective	Pricing-function approximation	Market-price prediction

Table 1.1: Scope of the project.

1.3 Research Question

The main research question is:

> *Can a feed-forward neural network accurately approximate the Black-Scholes pricing function for European call options?*

The purpose of the experiment is therefore not to outperform the analytical Black-Scholes formula. Since the formula is available in closed form, it remains the exact reference within the model assumptions. The relevant question is whether a neural network can learn the same pricing function with high accuracy and then act as a fast surrogate model at inference time.

The project compares three main pricing approaches:

- the analytical Black-Scholes formula, used as the ground truth;
- Monte Carlo simulation, used as a classical numerical benchmark;
- a feed-forward neural network, used as a supervised learning model and pricing surrogate.

Additional experiments consider feature engineering, activation functions, runtime performance, a Support Vector Regression baseline, and robustness to controlled label noise.

1.4 Contributions

The main contributions of the project are:

- construction of a reproducible synthetic Black-Scholes dataset;
- implementation of an analytical Black-Scholes benchmark and a Monte Carlo benchmark;
- training and evaluation of a feed-forward neural network pricing surrogate;
- comparison of model variants based on moneyness and activation functions;
- evaluation of runtime differences between analytical pricing, neural inference, and Monte Carlo simulation;
- inclusion of a reduced-scale SVR baseline;
- robustness experiments with controlled noisy Black-Scholes targets;
- modular Python implementation with tests, documentation, and reproducible experiment artifacts.

Overall, the project combines a classical financial model, a numerical benchmark, and supervised machine learning within a single reproducible experimental pipeline.

1.5 Report Structure

The report is organized as follows. Chapter 2 introduces the financial and mathematical background. Chapter 3 summarizes the machine learning concepts used in the project. Chapter 4 describes the dataset and experimental design. Chapter 5 presents the methodology and benchmark models. Chapter 6 describes the software implementation. Chapter 7 reports the experimental results. Chapter 8 discusses the interpretation and limitations of the results. Chapter 9 concludes and outlines possible future extensions.

2

Financial and Mathematical Background

2.1 European Call Options

A European call option gives its holder the right, but not the obligation, to buy an underlying asset at a fixed strike price K at maturity T. The term European means that the option can be exercised only at maturity, not before. If the terminal asset price is S_T, the payoff of a European call is:

$$\Phi(S_T) = \max(S_T - K, 0) = (S_T - K)^+. \tag{2.1}$$

The payoff is positive only when the option finishes in the money, namely when $S_T > K$. If $S_T \leq K$, exercising the option would not be convenient and the payoff is zero. The option value depends on the current asset price, the strike price, time to maturity, the risk-free rate, and the volatility of the underlying asset. General arbitrage-pricing principles for derivative securities can be introduced in discrete-time models before moving to continuous-time diffusion models [6, 5]. In this project, only European call options are considered.

Table 2.1 summarizes the main notation used throughout the report.

Symbol	Meaning
S_0	initial price of the underlying asset
S_t	price of the underlying asset at time t
K	strike price of the option
T	time to maturity, expressed in years
r	continuously compounded risk-free interest rate
σ	volatility of the underlying asset
W_t	standard Brownian motion
$\mathbb{Q}$	risk-neutral probability measure
$\Phi(S_T)$	payoff of the option at maturity
C_{BS}	analytical Black-Scholes call price
$\hat{C}_{MC}$	Monte Carlo estimate of the call price

Table 2.1: Main financial notation used in the report.

2.2 Stochastic Differential Equations

Continuous-time asset price models are often described by stochastic differential equations. An SDE combines a deterministic drift component with a random diffusion component driven by Brownian motion. This framework is standard in financial modeling because asset prices evolve

continuously but are subject to uncertainty [7]. In general form, an Itô diffusion can be written as:

$$dX_t = \mu(t, X_t)\, dt + \eta(t, X_t)\, dW_t, \tag{2.2}$$

where μ is the drift coefficient, η is the diffusion coefficient, and W_t is a Brownian motion. The term $\mu(t, X_t)\, dt$ describes the deterministic infinitesimal trend of the process, while $\eta(t, X_t)\, dW_t$ describes the random infinitesimal shock. In financial models, this decomposition is useful because asset prices display both systematic growth components and random fluctuations.

2.3 Geometric Brownian Motion

In the Black-Scholes model, the underlying asset follows a geometric Brownian motion. Under the risk-neutral measure $\mathbb{Q}$, the discounted asset price is a martingale; equivalently, the asset drift in the pricing dynamics is the risk-free rate r. The dynamics used for pricing are:

$$dS_t = rS_t dt + \sigma S_t dW_t, \tag{2.3}$$

where S_t is the asset price at time t, r is the risk-free interest rate, σ is the constant volatility, and W_t is a standard Brownian motion. The multiplicative structure of the SDE means that both the drift and the volatility scale with the current asset price. As a consequence, if $S_0 > 0$, the asset price remains positive under the model.

The solution at maturity T is:

$$S_T = S_0 \exp\left[\left(r - \frac{1}{2}\sigma^2\right) T + \sigma\sqrt{T} Z\right], \qquad Z \sim \mathcal{N}(0, 1). \tag{2.4}$$

This explicit terminal distribution is important for the implementation. It allows Monte Carlo simulation to sample S_T directly, without simulating all intermediate time steps of the path.

2.4 Black-Scholes Model

The Black-Scholes model assumes frictionless markets, constant volatility, a constant risk-free rate, no arbitrage, and continuous trading [1, 7]. Under these assumptions, the price of a European option can be derived from risk-neutral valuation. The main idea is that, in an arbitrage-free complete market, the option price is equal to the discounted expected payoff under the risk-neutral measure.

For a European call, the value is the discounted expected payoff:

$$C = e^{-rT} \mathbb{E}^{\mathbb{Q}} \left[\max(S_T - K, 0)\right]. \tag{2.5}$$

> *In this project, Black-Scholes is not treated as an empirical description of all market prices. It is used as a controlled mathematical pricing function that provides exact target values for supervised learning.*

2.5 Black-Scholes Formula

The analytical Black-Scholes price of a European call option is:

$$C_{BS} = S_0 N(d_1) - K e^{-rT} N(d_2), \tag{2.6}$$

where $N(\cdot)$ is the cumulative distribution function of a standard normal random variable and:

$$d_1 = \frac{\ln(S_0/K) + \left(r + \frac{1}{2}\sigma^2\right) T}{\sigma\sqrt{T}}, \tag{2.7}$$

$$d_2 = d_1 - \sigma\sqrt{T}. \tag{2.8}$$

In this project, C_{BS} is used both as the supervised learning target and as the reference value for evaluation. This is what makes the experiment controlled: the true target function is known analytically. The neural network is therefore trained to approximate the mapping:

$$f_{BS} : (S_0, K, T, r, \sigma) \mapsto C_{BS}. \tag{2.9}$$

This formulation turns option pricing into a supervised regression problem. Each synthetic observation contains a vector of financial parameters as input and the corresponding analytical Black-Scholes price as target.

2.6 Monte Carlo Pricing

Monte Carlo simulation approximates the risk-neutral expectation by generating many realizations of S_T [3]:

$$\hat{C}_{MC} = e^{-rT} \frac{1}{M} \sum_{i=1}^{M} \max(S_T^{(i)} - K, 0). \tag{2.10}$$

For sufficiently large M, $\hat{C}_{MC}$ converges to C_{BS} by the law of large numbers. The statistical error decreases at the standard Monte Carlo rate:

$$\mathrm{SE}(\hat{C}_{MC}) = \frac{e^{-rT}}{\sqrt{M}} \,\mathrm{Std}\left(\max(S_T - K, 0)\right). \tag{2.11}$$

This $M^{-1/2}$ convergence rate explains why Monte Carlo can become expensive when high accuracy is required. In this project, Monte Carlo is used as a numerical benchmark and as a comparison point for the neural network. Since the terminal distribution of S_T is known exactly under Black-Scholes, the implementation simulates terminal prices directly rather than discretizing the full path. This choice is both mathematically correct under the Black-Scholes assumptions and computationally more efficient than Euler discretization for the same model.

3

Machine Learning Background

3.1 Supervised Learning for Regression

The pricing task is formulated as a supervised regression problem. Each observation consists of option and market parameters, and the target is the corresponding Black-Scholes call price. Given a dataset $\{(x_i, y_i)\}_{i=1}^{n}$, where x_i is a vector of input features and y_i is a scalar target price, the goal is to learn a parametrized function f_θ such that:

$$f_\theta(x_i) \approx y_i. \tag{3.1}$$

In this project, the target y_i is not observed from markets. It is generated by the analytical Black-Scholes formula, which allows direct measurement of approximation error. The clean supervised learning problem can therefore be written as:

$$x_i = (S_{0,i}, K_i, T_i, r_i, \sigma_i), \qquad y_i = C_{BS}(S_{0,i}, K_i, T_i, r_i, \sigma_i). \tag{3.2}$$

For the final feature set, the moneyness ratio S_0/K is also provided to the model as an engineered input. In that case:

$$x_i = (S_{0,i}, K_i, T_i, r_i, \sigma_i, S_{0,i}/K_i). \tag{3.3}$$

The model parameters are estimated by minimizing a regression loss over the training data. For a neural network trained with mean squared error, the empirical objective is:

$$\mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^{n} \left(f_\theta(x_i) - y_i\right)^2. \tag{3.4}$$

This objective penalizes large pricing errors more strongly than small errors, which is appropriate for learning a smooth pricing surface. During evaluation, however, the project also reports metrics in original price units, such as MAE and RMSE, because they are easier to interpret financially.

> *In the main clean setting, the supervised learning target is not a noisy market quote. It is the exact Black-Scholes price, so the experiment measures approximation quality rather than market prediction ability.*

3.2 Neural Networks as Function Approximators

Feed-forward neural networks are flexible nonlinear function approximators. Under general assumptions, sufficiently large feed-forward networks can approximate continuous functions on compact domains [2, 4]. This makes them natural candidates for approximating a deterministic pricing function such as Black-Scholes.

The purpose of the neural network is not to replace the analytical formula when that formula is available. Rather, the Black-Scholes setting provides a clean benchmark for studying the surrogate-model idea before moving to models where closed-form prices may not exist. The relevant question is therefore whether a neural network can learn the shape of the pricing surface over a specified domain of financial parameters.

For a feed-forward neural network with L layers, the computation can be described recursively as:

$$h^{(0)} = x, \tag{3.5}$$

$$h^{(\ell)} = \phi\left(W^{(\ell)}h^{(\ell-1)} + b^{(\ell)}\right), \qquad \ell = 1, \ldots, L-1, \tag{3.6}$$

$$f_\theta(x) = W^{(L)}h^{(L-1)} + b^{(L)}. \tag{3.7}$$

Here $W^{(\ell)}$ and $b^{(\ell)}$ are trainable weights and biases, while ϕ is a nonlinear activation function. The final layer is linear because option pricing is treated as a scalar regression problem.

3.3 Feed-Forward Neural Network Architecture

The model used in the project is a Multi-Layer Perceptron. It receives numerical input features and returns one scalar output, the predicted option price.

The baseline architecture is:

$$5 \to 64 \to 64 \to 32 \to 1, \tag{3.8}$$

where the five inputs are (S_0, K, T, r, σ). After intermediate experiments, the selected final configuration uses six input features, including moneyness, and SiLU hidden activations:

$$6 \to 64 \to 64 \to 32 \to 1. \tag{3.9}$$

The final neural network approximates:

$$f_\theta(S_0, K, T, r, \sigma, S_0/K) \approx C_{BS}. \tag{3.10}$$

The architecture is intentionally simple. The goal is not to design a highly specialized neural architecture, but to test whether a standard fully connected network can approximate a known pricing function with high accuracy. The implementation builds the MLP dynamically from the selected input dimension, hidden layer sizes, and activation function:

```python
class PricingMLP(nn.Module):
    def __init__(
        self,
        input_dim: int = 5,
        hidden_layers: tuple[int, ...] = (64, 64, 32),
        activation: str = "relu",
    ):
        super().__init__()
        layers: list[nn.Module] = []
        previous_dim = input_dim

        for hidden_dim in hidden_layers:
            layers.append(nn.Linear(previous_dim, hidden_dim))
            layers.append(make_activation(activation))
            previous_dim = hidden_dim

        layers.append(nn.Linear(previous_dim, 1))
        self.net = nn.Sequential(*layers)
```

Listing 3.1: Core PyTorch implementation of the feed-forward pricing network.

This design makes the architecture configurable without duplicating model code. For example, the same class can represent both the baseline five-feature model and the final six-feature model with moneyness.

3.4 Support Vector Regression

Support Vector Regression is included as an additional classical machine learning baseline [8]. The project uses SVR with an RBF kernel, which is able to model nonlinear relationships in tabular data.

SVR is evaluated on reduced datasets because kernel methods generally scale less favorably than neural networks as the number of training observations increases. For this reason, the SVR experiments should be interpreted as a classical reference point rather than as a full-scale replacement for the neural network pipeline. The RBF kernel used in the benchmark has the form:

$$K(x, x') = \exp\left(-\gamma \|x - x'\|^2\right), \tag{3.11}$$

where γ controls the locality of the kernel. In SVR, errors inside an ϵ-insensitive tube are not penalized, while larger deviations contribute to the loss. This makes SVR a well-established classical baseline for nonlinear regression, but the computational cost increases substantially as the number of training samples grows.

Table 3.1 summarizes the different roles of the neural network and SVR in the project.

Aspect	Neural network	SVR
Role	main pricing surrogate	classical ML baseline
Scale	full dataset	reduced datasets
Nonlinearity	hidden activations	RBF kernel
Training objective	MSE optimization	ϵ-insensitive regression
Main limitation	architecture and training choices	poor scaling with sample size
Interpretation	final surrogate candidate	reference comparison

Table 3.1: Conceptual comparison between the neural network and SVR models used in the project.

3.5 Pricing Surrogate Models

A pricing surrogate is a model trained to approximate a pricing function. Once trained, the surrogate can produce prices quickly for new inputs. This is useful when the original pricing method is computationally expensive, for example when repeated Monte Carlo simulations are required.

In the present Black-Scholes setting, the analytical formula remains the fastest and most exact method. The relevant surrogate comparison is therefore mainly between neural network inference and Monte Carlo simulation. A surrogate model is useful only if two conditions are met:

- its approximation error is small enough for the intended use;
- its inference time is significantly lower than the numerical method it aims to replace.

For this reason, the project evaluates both prediction accuracy and runtime performance. The neural network is trained offline, but after training it can price a large batch of options with a single forward pass through the model. This is the main computational advantage being tested.

4

Dataset and Experimental Design

4.1 Synthetic Dataset Generation

The dataset is generated by randomly sampling option and market parameters from fixed ranges. Each parameter is sampled independently from a uniform distribution over the interval specified in the experiment configuration. For the primitive Black-Scholes feature set, each observation contains:

$$x = (S_0, K, T, r, \sigma), \tag{4.1}$$

and the target is:

$$y = C_{BS}(S_0, K, T, r, \sigma). \tag{4.2}$$

Using synthetic data is a methodological choice. The project studies whether a neural network can approximate a known mathematical pricing function, not whether it can fit noisy market option prices. With synthetic Black-Scholes data, the ground truth is exact and reproducible. This makes the experimental question well defined: if the model makes an error, that error is an approximation error with respect to a known target function.

> *Synthetic data are used not because market data are unimportant, but because the research question is about learning the Black-Scholes pricing function itself. Real exchange-traded option prices would mix this function with bid-ask spreads, liquidity effects, dividends, and implied-volatility structure.*

The dataset-generation code follows this idea directly. The random number generator is seeded, option parameters are sampled from the configured ranges, and labels are computed using the analytical Black-Scholes implementation:

```python
def generate_synthetic_dataset(config: DatasetConfig) -> pd.DataFrame:
    rng = np.random.default_rng(config.seed)
    n = config.n_samples

    data = {
        "s0": rng.uniform(config.s0_min, config.s0_max, size=n),
        "k": rng.uniform(config.k_min, config.k_max, size=n),
        "t": rng.uniform(config.t_min, config.t_max, size=n),
        "r": rng.uniform(config.r_min, config.r_max, size=n),
        "sigma": rng.uniform(config.sigma_min, config.sigma_max, size=n),
    }
    data[TARGET_COLUMN] = call_price(
        data["s0"], data["k"], data["t"], data["r"], data["sigma"]
    )
    data[MONEYNESS_COLUMN] = data["s0"] / data["k"]
    return pd.DataFrame(data)
```

Listing 4.1: Core synthetic dataset generation logic.

4.2 Input Variables and Target

The parameter ranges used to generate the dataset are reported in Table 4.1.

Variable	Description	Range
S_0	initial underlying price	$[50, 150]$
K	strike price	$[50, 150]$
T	maturity in years	$[0.1, 2]$
r	risk-free rate	$[0, 0.05]$
σ	volatility	$[0.1, 0.6]$

Table 4.1: Parameter ranges used to generate the synthetic dataset.

The target variable is the European call option price computed using the analytical Black-Scholes formula. Therefore, each supervised learning observation has the form:

$$(S_0, K, T, r, \sigma) \mapsto C_{BS}. \tag{4.3}$$

The generated dataset also stores moneyness as an additional column. This column is always available for diagnostics, and it can optionally be included as a model input through the selected feature set. Table 4.2 summarizes the columns saved in the generated CSV file.

Column	Meaning	Role
`s0`	initial underlying price S_0	base input
`k`	strike price K	base input
`t`	maturity T	base input
`r`	risk-free rate r	base input
`sigma`	volatility σ	base input
`call_price`	analytical Black-Scholes call price	target
`moneyness`	ratio S_0/K	diagnostic or engineered input

Table 4.2: Columns contained in the clean synthetic dataset.

4.3 Feature Engineering: Moneyness

The final model augments the input vector with moneyness:

$$\text{moneyness} = \frac{S_0}{K}. \tag{4.4}$$

The final input vector is:

$$x_{\text{final}} = (S_0, K, T, r, \sigma, S_0/K). \tag{4.5}$$

Moneyness does not add new information beyond S_0 and K, but it exposes a financially meaningful relationship directly to the model. Intermediate experiments showed that this engineered feature can improve absolute-error metrics. The code keeps the feature choice explicit through two named feature sets:

- `base`: uses only `s0`, `k`, `t`, `r`, and `sigma`;
- `with_moneyness`: adds the `moneyness` column to the primitive inputs.

This design allows the same training pipeline to compare the baseline and engineered-feature settings without changing the rest of the experiment.

4.4 Train/Validation/Test Split

The dataset is split into training, validation, and test sets. The training set is used to optimize the model parameters. The validation set is used for early stopping and model selection during training. The test set is kept separate and is used only for final evaluation.

All splits are controlled by fixed random seeds. This makes the reported results reproducible and ensures that different model variants can be compared under consistent conditions. The default split proportions are reported in Table 4.3.

Split	Default fraction	Purpose
Training	70%	parameter optimization
Validation	15%	early stopping and model selection
Test	15%	final out-of-sample evaluation

Table 4.3: Default dataset split used by the neural-network training pipeline.

Operationally, the test set is reserved first. The remaining observations are then split into training and validation sets. This prevents the test set from influencing early stopping or model selection.

4.5 Clean and Noisy Target Settings

The main experiment uses clean Black-Scholes targets. The project also includes a controlled noisy-target setting used for robustness analysis. In that setting, the input variables are unchanged, while the training labels are perturbed.

For each clean price C_{BS}, the noisy target is generated as:

$$\widetilde{C} = \max\left(C_{\mathrm{BS}} + \varepsilon, 0\right), \qquad \varepsilon \sim \mathcal{N}\left(0, \left(\ell \max(C_{\mathrm{BS}}, 1)\right)^2\right), \tag{4.6}$$

where ℓ is the noise level. The implementation also stores the clean price, noisy price, sampled noise, and noise standard deviation in separate columns. The standard **`call_price`** column is overwritten by the noisy target only inside the noisy-target experiment, so the existing training code can be reused without changing the main clean pipeline. The evaluation target remains the clean analytical Black-Scholes price. This experiment measures robustness to imperfect labels while preserving a known ground truth.

```
noise_std = config.level * np.maximum(clean_prices, config.price_floor)
sampled_noise = rng.normal(loc=0.0, scale=noise_std)
noisy_prices = np.maximum(clean_prices + sampled_noise, 0.0)

noisy_df[CLEAN_TARGET_COLUMN] = clean_prices
noisy_df[NOISE_STD_COLUMN] = noise_std
noisy_df[PRICE_NOISE_COLUMN] = noisy_prices - clean_prices
noisy_df[NOISY_TARGET_COLUMN] = noisy_prices
noisy_df[TARGET_COLUMN] = noisy_prices
```

Listing 4.2: Controlled noisy-target generation used in robustness experiments.

Table 4.4 reports the additional columns produced in the noisy-target setting.

Column	Meaning
`clean_call_price`	original analytical Black-Scholes price
`noisy_call_price`	perturbed non-negative training label
`price_noise`	realized additive noise after clipping
`price_noise_std`	standard deviation used to sample the noise

Table 4.4: Additional columns stored in noisy-target robustness datasets.

4.6 Reproducibility

Reproducibility is handled through fixed random seeds and explicit configuration objects. Command-line options define each run, and the corresponding artifacts are saved to disk. Generated datasets and model checkpoints are stored in reproducible output directories. Selected final metrics and figures are tracked in the repository. The main clean dataset is saved as **`synthetic_options.csv`** in the configured data directory. Noisy-target datasets are written to separate experiment directories, so they do not overwrite the main clean dataset.

The project records three levels of reproducibility information:

- the random seeds used for dataset generation, data splitting, training, Monte Carlo simulation, and noise generation;
- the explicit configuration values used by each run;
- the generated artifacts, including datasets, metrics, figures, model checkpoints, and fitted scalers.

This makes it possible to inspect not only the final numerical results, but also the exact experimental setup that produced them.

5

Methodology

5.1 Experimental Pipeline

The methodology follows a controlled supervised-learning pipeline. First, synthetic Black-Scholes contracts are generated. Second, analytical Black-Scholes prices are used as reference labels. Third, machine-learning models are trained to approximate the pricing map. Finally, predictions are evaluated against the analytical reference and compared with Monte Carlo simulation.

The complete experimental logic can be summarized as:

$$(S_0, K, T, r, \sigma) \xrightarrow{\text{Black-Scholes}} C_{BS} \xrightarrow{\text{supervised learning}} \hat{C}. \tag{5.1}$$

The central evaluation question is whether $\hat{C}$ is close to C_{BS} on held-out test observations. The test set is never used during training or validation.

> *The analytical Black-Scholes formula defines the ground truth. Monte Carlo and machine-learning models are evaluated by how accurately and efficiently they reproduce that reference.*

5.2 Baseline Black-Scholes Pricing

The analytical Black-Scholes formula is the reference pricing method. It has two roles in the methodology:

- it generates the supervised learning labels in the synthetic dataset;
- it provides the exact reference values used to compute prediction errors.

Because the formula is available in closed form, it provides an exact ground truth for the controlled experiment. The baseline price for each option is:

$$C_{BS}(S_0, K, T, r, \sigma) = S_0 N(d_1) - K e^{-rT} N(d_2). \tag{5.2}$$

This reference is not treated as an empirical market price. It is a deterministic mathematical target under the assumptions of the Black-Scholes model.

5.3 Monte Carlo Benchmark

Monte Carlo simulation is used as a numerical benchmark. For each option, terminal prices are sampled from the exact Black-Scholes terminal distribution and discounted payoffs are averaged. For M simulated paths, the estimator is:

$$\hat{C}_{MC} = e^{-rT} \frac{1}{M} \sum_{j=1}^{M} (S_T^{(j)} - K)^+. \tag{5.3}$$

The implementation does not simulate the whole trajectory of the underlying asset. Since the option is European and path-independent, it is enough to sample the terminal value:

$$S_T^{(j)} = S_0 \exp\left[\left(r - \frac{1}{2}\sigma^2\right) T + \sigma\sqrt{T} Z_j\right], \qquad Z_j \sim \mathcal{N}(0,1). \tag{5.4}$$

The final Monte Carlo benchmark uses 50,000 simulated paths on a deterministic subset of 512 test options. The subset restriction keeps the benchmark computationally bounded while still providing a meaningful comparison with a classical simulation-based pricing method. The implementation is batched over both options and paths, so memory usage is controlled by the product of option batch size and path batch size.

```
for path_start in range(0, n_paths, path_batch_size):
    current_paths = min(path_batch_size, n_paths - path_start)
    z = rng.standard_normal(size=(end - start, current_paths))
    drift = (r[sl, None] - 0.5 * sigma[sl, None] ** 2) * t[sl, None]
    diffusion = sigma[sl, None] * np.sqrt(t[sl, None]) * z
    terminal_price = s0[sl, None] * np.exp(drift + diffusion)
    payoff_sum += call_payoff(terminal_price, k[sl, None]).sum(axis=1)
```

Listing 5.1: Batched exact-terminal Monte Carlo simulation under Black-Scholes dynamics.

5.4 Neural Network Model

The neural network is a feed-forward MLP for scalar regression. The selected final configuration uses the engineered feature set with moneyness and SiLU activations. Its input-output map is:

$$f_\theta(S_0, K, T, r, \sigma, S_0/K) \approx C_{BS}. \tag{5.5}$$

Table 5.1 reports the main neural-network configuration used in the final experiment.

Component	Final setting
Input features	$(S_0, K, T, r, \sigma, S_0/K)$
Hidden layers	64, 64, 32
Hidden activation	SiLU
Output dimension	one scalar call price
Loss	mean squared error
Optimizer	Adam
Batch size	1024
Maximum epochs	200
Early stopping patience	20 epochs
Weight decay	10^{-6}

Table 5.1: Final neural-network configuration used in the main experiment.

The optimized loss is the mean squared error:

$$\mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^{n} \left(f_\theta(x_i) - y_i \right)^2 . \tag{5.6}$$

5.5 Training Procedure

Training uses mini-batch gradient-based optimization. The procedure contains four important preprocessing and validation choices.

First, input features are standardized using `StandardScaler`. For each feature j, the transformed value is:

$$x_{ij}^{\text{scaled}} = \frac{x_{ij} - \mu_j}{s_j}, \tag{5.7}$$

where μ_j and s_j are computed only on the training set. The same fitted scaler is then applied to validation and test data.

Second, the target price is also standardized during training. This improves the numerical conditioning of MSE optimization because option prices can have a wider scale than the normalized inputs. Predictions are transformed back to original price units before metric computation.

Third, early stopping is based on validation loss. At each epoch, the model state with the best validation loss is saved. If validation loss does not improve for the configured patience window, training stops and the best state is restored.

Fourth, predictions are clipped at zero after inverse target scaling. This enforces the financial constraint that European call prices cannot be negative.

```python
if val_loss < best_val_loss:
    best_val_loss = val_loss
    best_state = {
        key: value.detach().cpu().clone()
        for key, value in model.state_dict().items()
    }
    epochs_without_improvement = 0
else:
    epochs_without_improvement += 1

if epochs_without_improvement ≥ config.patience:
    break
```

Listing 5.2: Validation-based early stopping used during neural-network training.

5.6 Evaluation Metrics

All predictive models are evaluated against reference prices on held-out test data. Let y_i denote the analytical Black-Scholes price and $\hat{y}_i$ the predicted or simulated price. The main metrics are:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|, \tag{5.8}$$

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}, \tag{5.9}$$

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}. \tag{5.10}$$

The project also reports MAPE, but only for options whose theoretical price is greater than 1:

$$\text{MAPE}_{y>1} = \frac{100}{|\mathcal{I}|}\sum_{i\in\mathcal{I}}\left|\frac{y_i - \hat{y}_i}{y_i}\right|, \qquad \mathcal{I} = \{i : |y_i| > 1\}. \tag{5.11}$$

This restriction avoids unstable percentage errors for options whose theoretical price is close to zero. MAE and RMSE are still computed on the full test set.

5.7 Runtime Benchmark

The runtime benchmark separates training time from pricing time. This distinction is important because a neural network has an upfront training cost, while its surrogate value is measured during repeated inference.

The benchmark compares:

- vectorized analytical Black-Scholes pricing;
- neural network inference after training;
- Monte Carlo pricing.

The reported throughput is computed as:

$$\text{throughput} = \frac{\text{number of priced options}}{\text{elapsed seconds}}. \tag{5.12}$$

The neural-network timing uses a warmup forward pass before repeated inference timings. This avoids counting one-off PyTorch setup overhead as part of steady-state inference. Monte Carlo timings are repeated fewer times because each run is substantially more expensive.

5.8 SVR Baseline

The project includes a reduced-scale Support Vector Regression baseline with an RBF kernel. The SVR benchmark uses 5,000 synthetic samples per run and is repeated over three random seeds. The same engineered feature set used by the final neural network, including moneyness, is used for SVR.

Both input features and target prices are standardized before fitting. Predictions are transformed back to original price units before computing metrics. SVR is kept on reduced datasets because kernel methods scale less favorably than neural networks.

The SVR baseline is not intended to replace the neural-network pipeline. Its role is to provide a classical nonlinear regression reference, useful for checking whether the neural network performance is competitive with a standard machine-learning model on smaller datasets.

5.9 Robustness Experiments with Noisy Targets

The robustness experiments perturb only the training labels, not the input variables. Models are trained on noisy Black-Scholes prices and evaluated against clean analytical Black-Scholes prices. This separates label-noise robustness from market-data modeling.

The neural network noisy-target experiment uses 50,000 synthetic contracts per noise level. The same noisy-target mechanism is also applied to the reduced-scale SVR baseline, using 5,000 contracts per run and three random seeds.

Table 5.2 summarizes the main experimental components.

Component	Dataset size	Purpose
Final neural network	100,000	main pricing surrogate
Monte Carlo benchmark	512 test options	numerical benchmark
Runtime benchmark	1,000 priced options	inference-speed comparison
SVR benchmark	5,000 per seed	classical ML baseline
Noisy-target NN	50,000 per noise level	label-noise robustness
Noisy-target SVR	5,000 per seed	noisy-label classical baseline

Table 5.2: Main experimental components and their methodological role.

These experiments are controlled label-noise tests. They should not be interpreted as real market-price modeling, because real exchange-traded options include additional effects such as bid-ask spreads, liquidity, dividends, and implied-volatility structure.

6

Software Implementation

6.1 Implementation Goals

The project is implemented as a small Python package rather than as a single script. This design choice keeps the mathematical components, training logic, benchmarks, plotting functions, and command-line interfaces separated. The main implementation goals are:

- reproducibility: experiments are controlled by explicit configuration objects and random seeds;
- modularity: each file has a focused responsibility;
- efficiency: vectorized NumPy operations, batched Monte Carlo simulation, and mini-batch neural-network training are used;
- inspectability: metrics, figures, datasets, models, scalers, and configuration snapshots are saved to disk.

The main libraries are NumPy, pandas, SciPy, scikit-learn, matplotlib, and PyTorch.

> *The codebase is organized around a reusable scientific package. Command-line scripts are thin entry points that configure and execute package functions.*

6.2 Repository Structure

The reusable scientific and machine-learning logic is contained in `src/nn_option_pricing/`. The files in `scripts/` are command-line entry points that build configurations and call functions from the package.

```
src/nn_option_pricing/
    black_scholes.py          # analytical formula and payoff
    config.py                 # experiment configuration dataclasses
    dataset.py                # synthetic data generation and storage
    evaluation.py             # regression metrics
    model.py                  # PyTorch feed-forward neural network
    monte_carlo.py            # efficient Monte Carlo simulation
    noise.py                  # controlled target-noise generation
    noisy_experiment.py       # neural-network noisy-target experiment
    noisy_svr_experiment.py   # noisy-target SVR benchmark
    pipeline.py               # end-to-end experiment
    plots.py                  # visualizations
    svr.py                    # reduced-scale SVR benchmark
    training.py               # training loop, scaling, early stopping
scripts/
    run_experiment.py             # main experiment CLI
    benchmark_runtime.py          # runtime benchmark CLI
    run_svr_benchmark.py          # SVR benchmark CLI
    run_noisy_targets_experiment.py
    run_noisy_svr_benchmark.py
tests/
    test_black_scholes.py
    test_dataset.py
    test_model.py
    test_monte_carlo.py
    test_noise.py
    test_noisy_svr.py
    test_pipeline.py
    test_svr.py
```

Listing 6.1: Main source, script, and test structure.

Table 6.1 summarizes the role of the most important implementation modules.

Module	Main responsibility	Used by
`black_scholes.py`	analytical pricing and payoff	dataset, benchmarks
`dataset.py`	synthetic data generation	pipeline, SVR experiments
`training.py`	scaling, training, prediction	main and noisy NN experiments
`monte_carlo.py`	batched Monte Carlo estimator	pipeline, runtime benchmark
`evaluation.py`	regression metrics	all experiments
`pipeline.py`	end-to-end main run	`run_experiment.py`
`svr.py`	reduced-scale SVR benchmark	`run_svr_benchmark.py`
`noise.py`	controlled label noise	noisy-target experiments

Table 6.1: Main Python modules and their responsibilities.

6.3 Configuration System

The project uses frozen dataclasses to keep experiment parameters explicit. The configuration layer includes dataset ranges, training hyperparameters, Monte Carlo settings, and output paths. This avoids hard-coded constants scattered across the codebase and makes experiments easier to reproduce.

The top-level configuration groups the main components:

```python
@dataclass(frozen=True)
class ExperimentConfig:
    dataset: DatasetConfig = DatasetConfig()
    training: TrainingConfig = TrainingConfig()
    monte_carlo: MonteCarloConfig = MonteCarloConfig()
    paths: PathConfig = PathConfig()
```

Listing 6.2: Top-level experiment configuration object.

Each experiment saves a configuration snapshot in JSON format. Since dataclasses and `Path` objects are not directly JSON-serializable, the helper `config_to_dict` recursively converts configuration objects into JSON-compatible values. This makes it possible to recover the exact settings used to produce a result.

6.4 Command-Line Interfaces

The project is executed through command-line scripts. The main experiment is launched by the script `run_experiment.py`. It parses command-line arguments, builds an `ExperimentConfig`, runs the pipeline, and prints the resulting metrics.

```python
config = ExperimentConfig(
    dataset=DatasetConfig(n_samples=args.n_samples, seed=args.seed),
    training=TrainingConfig(
        seed=args.seed,
        feature_set=args.feature_set,
        max_epochs=args.max_epochs,
        batch_size=args.batch_size,
        learning_rate=args.learning_rate,
        activation=args.activation,
    ),
    monte_carlo=MonteCarloConfig(
        n_paths=args.mc_n_paths,
        evaluation_samples=args.mc_evaluation_samples,
    ),
    paths=make_path_config(data_dir=args.data_dir, output_dir=args.output_dir),
)
```

Listing 6.3: Construction of the main experiment configuration from CLI arguments.

The final experiment is run through:

```
.venv/bin/python scripts/run_experiment.py \
    --n-samples 100000 \
    --max-epochs 200 \
    --batch-size 1024 \
    --mc-n-paths 50000 \
    --mc-evaluation-samples 512 \
    --feature-set with_moneyness \
    --activation silu \
    --seed 42 \
    --data-dir data/final_improved \
    --output-dir outputs/final_improved
```

Listing 6.4: Command used for the final experiment.

Additional scripts run the runtime benchmark, the SVR benchmark, the noisy-target neural-

network experiment, and the noisy-target SVR benchmark. This keeps the main experiment clean while still allowing extensions to be reproduced independently.

6.5 End-to-End Pipeline

The main pipeline connects all core components. It is implemented in `pipeline.py` and performs the following steps:

1. create output directories;
2. save the experiment configuration;
3. generate and save the synthetic dataset;
4. train the neural network;
5. predict on the held-out test set;
6. compute neural-network metrics;
7. save the trained model, input scaler, and target scaler;
8. run the Monte Carlo benchmark on a deterministic test subset;
9. save metrics and diagnostic plots.

This structure ensures that each run leaves behind enough information to inspect, reproduce, and discuss the experiment.

Table 6.2 reports the main artifacts produced by the pipeline.

Artifact	Typical location	Purpose
Dataset CSV	`data/` ... `/synthetic_options.csv`	generated observations
Configuration JSON	`outputs/` ... `/experiment_config.json`	reproducibility
Model checkpoint	`outputs/` ... `/model.pt`	trained network state
Input scaler	`outputs/` ... `/scaler.joblib`	feature preprocessing
Target scaler	`outputs/` ... `/target_scaler.joblib`	inverse price scaling
Metrics JSON	`outputs/` ... `/metrics/`	quantitative evaluation
Figures	`outputs/` ... `/figures/`	diagnostic plots

Table 6.2: Main artifacts produced by a complete experiment run.

6.6 Testing and Documentation

The test suite checks both low-level mathematical functions and higher-level experiment behavior. This is important because numerical projects can fail silently if only final metrics are inspected.

Test area	Purpose
Black-Scholes tests	validate analytical pricing behavior
Dataset tests	check generated columns, ranges, and shape
Model tests	verify neural-network construction and outputs
Monte Carlo tests	check simulation behavior and convergence properties
Noise tests	validate controlled noisy-target generation
SVR tests	check reduced-scale benchmark execution
Noisy-SVR tests	check noisy-label SVR benchmark execution
Pipeline tests	run small end-to-end smoke tests

Table 6.3: Main test areas covered by the project.

The project also includes Sphinx documentation generated from narrative documentation and Python docstrings. This supports code readability and makes the implementation easier to inspect independently from the report.

6.7 Computational Efficiency

Several implementation choices are made for computational efficiency. The Black-Scholes formula and dataset generation are vectorized with NumPy. This allows prices for many contracts to be computed at once without explicit Python loops.

The Monte Carlo method uses the exact terminal distribution of geometric Brownian motion, avoiding unnecessary time discretization. Simulation is batched both over options and over paths, which keeps memory usage bounded. If B_o is the option batch size and B_p is the path batch size, the largest simulated terminal-price block has shape approximately:

$$B_o \times B_p. \tag{6.1}$$

The neural network training uses PyTorch `DataLoader`, mini-batch gradient descent, input standardization, and target scaling. Prediction is performed in batches under `torch.no_grad()`, so gradients are not stored during inference. These choices make the code suitable both for quick development runs and for larger experiments.

7

Experimental Results

7.1 Reading Guide

This chapter reports the empirical results obtained from the experiments described in the previous chapters. The main reference target is always the analytical Black-Scholes price C_{BS}. When a model produces an estimate $\hat{C}$, the reported errors measure the distance between $\hat{C}$ and C_{BS} on held-out observations.

The experiments are organized into four groups:

- the final neural-network experiment, trained on 100,000 synthetic options;
- the Monte Carlo and runtime comparisons, used as quantitative benchmarks;
- intermediate model-selection experiments, used to justify moneyness and SiLU;
- robustness and classical machine-learning checks, based on noisy targets and SVR.

The different experiments do not all have the same computational scale. The final neural network is the main experiment, while SVR benchmarks are intentionally run on reduced datasets because kernel methods scale less favorably with the number of training samples.

> *The main empirical conclusion is not that the neural network replaces the analytical Black-Scholes formula. Rather, the experiment shows that a supervised neural network can learn the Black-Scholes pricing map very accurately and can act as a fast pricing surrogate after training.*

7.2 Final Neural Network Performance

The selected final model uses the engineered moneyness feature and SiLU activations. It is trained on 100,000 synthetic contracts generated under the Black-Scholes framework, with a 70%/15%/15% train-validation-test split. The final supervised learning map is:

$$(S_0, K, T, r, \sigma, S_0/K) \mapsto \hat{C}_{NN}. \tag{7.1}$$

The results on the held-out test set are reported in Table 7.1.

Metric	Value
MAE	0.04290
RMSE	0.06208
R^2	0.999993
MAPE, prices > 1	0.4803%

Table 7.1: Final neural network metrics against analytical Black-Scholes prices.

The error levels are small in absolute price units. The coefficient of determination is very close to one, which means that almost all variability in the analytical Black-Scholes prices over the sampled domain is reproduced by the neural network. The MAPE value, computed only for options whose theoretical price is greater than 1, confirms that the relative error is also low in economically meaningful price regions.

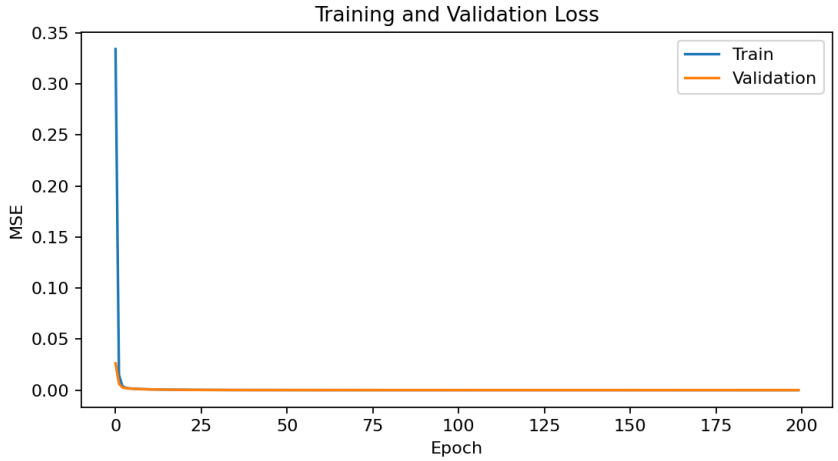


Figure 7.1: Training loss and validation loss for the final experiment.

Figure 7.1 shows that the optimization process is stable. The validation curve follows the training curve closely, which suggests that the model is not simply memorizing the training set. This is important because the goal is to approximate the pricing function on unseen parameter combinations, not only to fit the sampled contracts.

Figure 7.2: Analytical Black-Scholes prices against neural network predictions.

Figure 7.2 compares analytical prices with neural-network predictions. The points are tightly concentrated around the diagonal identity line, which visually confirms the high R^2 reported in Table 7.1.

7.3 Monte Carlo Comparison

Monte Carlo pricing is evaluated against the same analytical Black-Scholes reference, but on a deterministic subset of 512 test options for computational reasons. Each option is priced using 50,000 terminal samples. The comparison is therefore a numerical benchmark rather than a fully symmetric replacement for the neural-network test-set evaluation.

Metric	Value
MAE	0.08882
RMSE	0.15790
R^2	0.999951
MAPE, prices > 1	0.6483%

Table 7.2: Final Monte Carlo metrics against analytical Black-Scholes prices.

The Monte Carlo benchmark also reaches high accuracy, as expected with a large number of paths. However, Monte Carlo remains a stochastic estimator:

$$\hat{C}_{MC} = C_{BS} + \varepsilon_{MC}, \qquad \varepsilon_{MC} = O_P(M^{-1/2}), \tag{7.2}$$

where M is the number of simulated paths. Increasing M reduces sampling error, but also increases runtime. By contrast, once trained, the neural network produces deterministic forward-

pass predictions.

Method	MAE	RMSE	R^2	MAPE, prices > 1
Neural network	0.04290	0.06208	0.999993	0.4803%
Monte Carlo	0.08882	0.15790	0.999951	0.6483%

Table 7.3: Summary comparison of the final neural network and Monte Carlo benchmark.

Table 7.3 should be interpreted with care because the two methods are evaluated on different test-set sizes. Nevertheless, it shows that the neural network surrogate reaches at least the same order of accuracy as the Monte Carlo benchmark in this controlled setting.

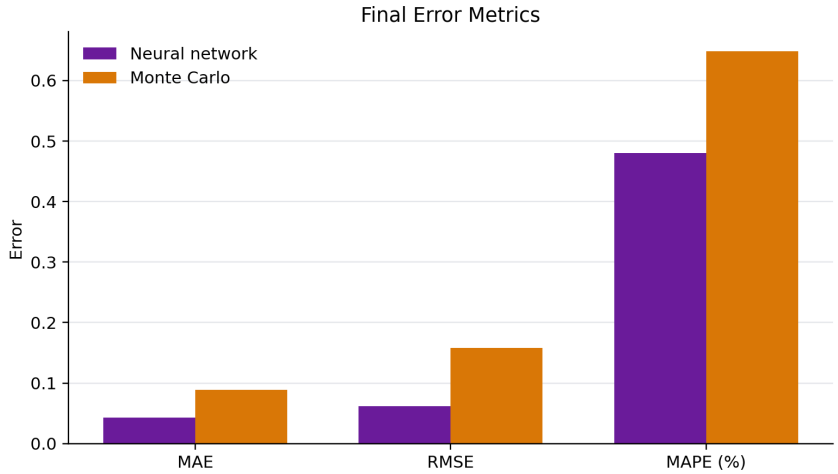


Figure 7.3: Final neural-network and Monte Carlo errors against analytical Black-Scholes prices.

Figure 7.3 provides a visual summary of the same comparison. The neural-network bars are lower for all reported error metrics, while the interpretation remains tied to the different evaluation protocols described above.

7.4 Feature Engineering and Activation Function Results

Before selecting the final configuration, intermediate experiments were used to test two design choices: adding the moneyness feature and changing the hidden-layer activation function. These experiments use 50,000 synthetic contracts and 100 training epochs.

7.4.1 Moneyness Feature

The moneyness experiment compares the base feature set

$$(S_0, K, T, r, \sigma) \tag{7.3}$$

with the engineered feature set

$$(S_0, K, T, r, \sigma, S_0/K). \tag{7.4}$$

Table 7.4 reports the results.

Experiment	MAE	RMSE	R^2	MAPE, prices > 1
Base + ReLU	0.17269	0.22973	0.999904	1.8687%
Moneyness + ReLU	0.15417	0.20930	0.999920	1.6670%

Table 7.4: Intermediate feature-engineering experiment.

Adding moneyness reduces MAE by approximately 10.7% in this controlled intermediate setting. The feature does not add new theoretical information, because it is computed from S_0 and K. However, it exposes a financially meaningful ratio directly to the model, which can simplify the regression problem.

7.4.2 Activation Functions

The activation-function experiment compares ReLU, Tanh, and SiLU using the same base feature set. The results are reported in Table 7.5.

Experiment	MAE	RMSE	R^2	MAPE, prices > 1
Base + ReLU	0.17269	0.22973	0.999904	1.8687%
Base + Tanh	0.11316	0.16541	0.999950	1.2878%
Base + SiLU	0.10775	0.15817	0.999954	1.2488%

Table 7.5: Intermediate activation-function experiment.

Both smooth activations outperform ReLU in this setting, with SiLU giving the lowest MAE and RMSE. This is consistent with the target being a smooth analytical pricing function over the sampled parameter domain.

7.4.3 Combined Configuration

The strongest intermediate candidate combines the moneyness feature with SiLU activations. Table 7.6 summarizes the comparison with the other intermediate configurations.

Experiment	MAE	RMSE	R^2	MAPE, prices > 1
Base + ReLU	0.17269	0.22973	0.999904	1.8687%
Moneyness + ReLU	0.15417	0.20930	0.999920	1.6670%
Base + SiLU	0.10775	0.15817	0.999954	1.2488%
Moneyness + SiLU	0.10254	0.15169	0.999958	1.3213%

Table 7.6: Combined feature and activation experiment on the intermediate setup.

The combined configuration gives the best MAE and RMSE among the intermediate experiments. It does not dominate every metric: the best MAPE in this table is obtained by Base + SiLU. For the final run, the combined configuration was selected because MAE and RMSE are the primary absolute-error metrics in price units.

7.5 Runtime Results

The runtime benchmark separates the initial training cost from pricing time after training. This distinction matters because a neural network is expensive to train once, but cheap to evaluate many times afterwards.

The benchmark uses the improved neural configuration, 50,000 synthetic training samples, 100 epochs, and 1,000 pricing queries. The Monte Carlo benchmark uses 20,000 paths. Results are reported in Table 7.7.

Method	Mean time for 1,000 options	Mean options/s
Black-Scholes formula	0.00091 s	1,101,094.82
Neural network inference	0.03190 s	31,350.77
Monte Carlo, 20,000 paths	1.28235 s	779.82

Table 7.7: Runtime benchmark for analytical, neural-network, and Monte Carlo pricing.

The analytical formula is the fastest method when it is available. The more relevant surrogate-model comparison is therefore between neural-network inference and Monte Carlo simulation. In this benchmark, after training, neural inference is approximately 40 times faster than Monte Carlo for the same 1,000 pricing queries. The trade-off is the initial training cost, which was approximately 359 seconds in the benchmark environment.

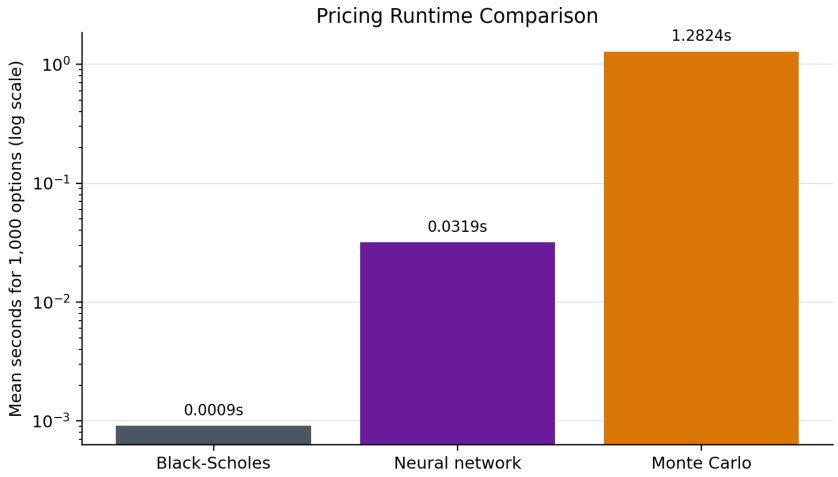


Figure 7.4: Mean pricing time for 1,000 options.

Figure 7.4 uses a logarithmic vertical axis because the three pricing methods operate on very different time scales. This makes the inference-time advantage over Monte Carlo visible without hiding the fact that the analytical Black-Scholes formula remains the fastest method in this specific setting.

7.6 SVR Benchmark Results

Support Vector Regression provides an additional classical machine-learning baseline. The experiment is repeated over three random seeds, using 5,000 synthetic contracts per run and the same engineered moneyness feature used by the final neural network. Input features and

target prices are standardized before fitting the SVR, and predictions are transformed back to price units before computing metrics.

Metric	Mean	Standard deviation
MAE	0.15088	0.00732
RMSE	0.23100	0.00592
R^2	0.999905	0.000005
MAPE, prices > 1	1.8253%	0.2422%
Fit time	12.3198 s	0.7977 s
Prediction time	0.0301 s	0.0029 s

Table 7.8: Reduced-scale SVR benchmark aggregated over three random seeds.

The SVR baseline approximates the Black-Scholes function well on reduced datasets, as shown by its high R^2. However, its absolute error is higher than that of the final neural network configuration. Because the SVR experiment uses smaller datasets, this result should be read as a classical baseline check rather than as a direct large-scale competition.

7.7 Noisy Target Robustness Results

The noisy-target experiment tests whether the neural network can still recover the clean analytical pricing function when the training labels are perturbed. For each clean price C_{BS}, the training target is:

$$\tilde{C} = \max\left(C_{BS} + \epsilon, 0\right), \qquad \epsilon \sim \mathcal{N}\left(0, \left[\alpha \max(C_{BS}, 1)\right]^2\right), \tag{7.5}$$

where α is the noise level. The model is trained on $\tilde{C}$ but evaluated against the clean C_{BS}. This keeps the experiment aligned with the original research question while adding controlled label noise. The 0% row in Table 7.9 is the clean-label control for this robustness setup, not a repetition of the final 100,000-sample experiment.

Noise level	MAE	RMSE	R^2	MAPE, prices > 1
0%	0.10254	0.15169	0.999958	1.3213%
1%	0.11084	0.16105	0.999953	1.3960%
5%	0.17827	0.24167	0.999893	2.0943%

Table 7.9: Noisy-target neural-network results evaluated against clean Black-Scholes prices.

Table 7.9 shows the expected degradation as label noise increases. The model remains accurate under 1% noise, while 5% noise produces a clearer increase in both MAE and RMSE. This supports a cautious robustness interpretation: the neural network can partially smooth noisy labels, but noisy supervision still reduces approximation quality.

The same noisy-target experiment is repeated with the reduced-scale SVR baseline. Aggregate metrics against clean Black-Scholes prices are reported in Table 7.10.

Noise level	MAE	RMSE	R^2	MAPE, prices > 1
0%	0.15088	0.23100	0.999905	1.8253%
1%	0.19284	0.30765	0.999830	1.8705%
5%	0.56185	1.05057	0.998025	3.4903%

Table 7.10: Noisy-target SVR benchmark evaluated against clean Black-Scholes prices.

SVR remains accurate under mild label noise, but it degrades more strongly at the 5% noise level. This comparison is useful because it shows that the noisy-target effect is not specific to the neural-network implementation. At the same time, the SVR experiment remains a reduced-scale benchmark and should not be interpreted as a replacement for the final neural-network experiment.

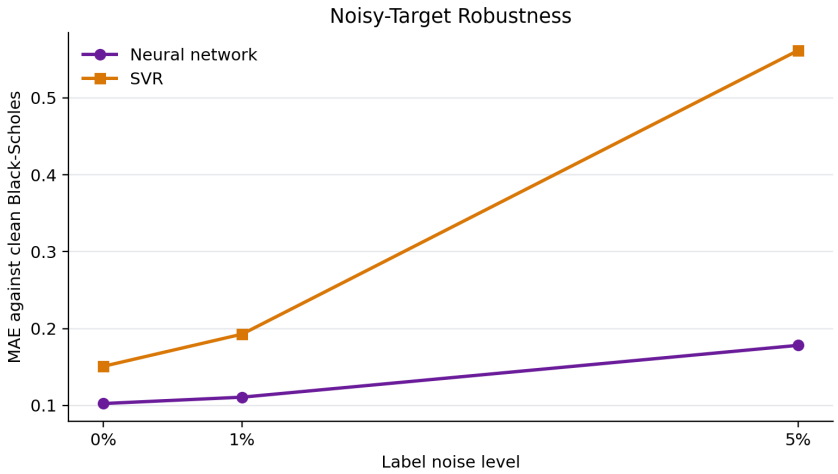


Figure 7.5: MAE against clean Black-Scholes prices under increasing label noise.

Figure 7.5 highlights the monotone degradation caused by increasing label noise. The neural-network curve remains below the SVR curve in this experiment, but the comparison should be interpreted as a robustness reference rather than as a fully symmetric benchmark because the two models are evaluated at different computational scales.

7.8 Error Analysis

Aggregate metrics are necessary but not sufficient. Two models with similar MAE may behave differently across moneyness, maturity, or volatility regions. For this reason, the final experiment includes diagnostic plots of prediction errors.

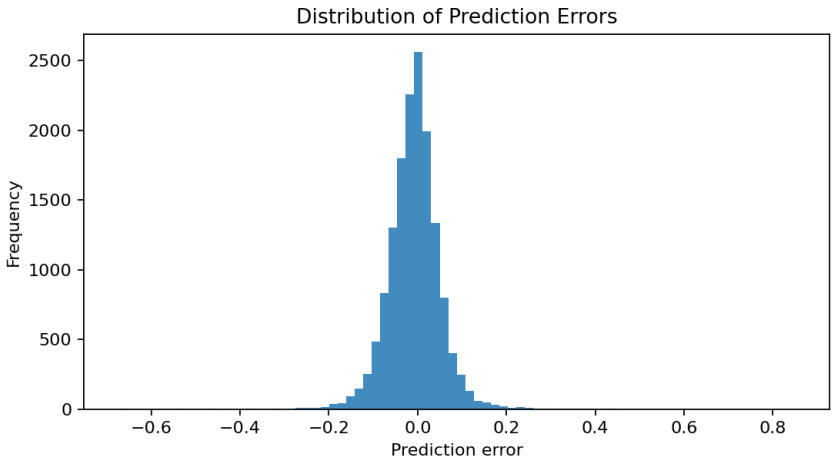
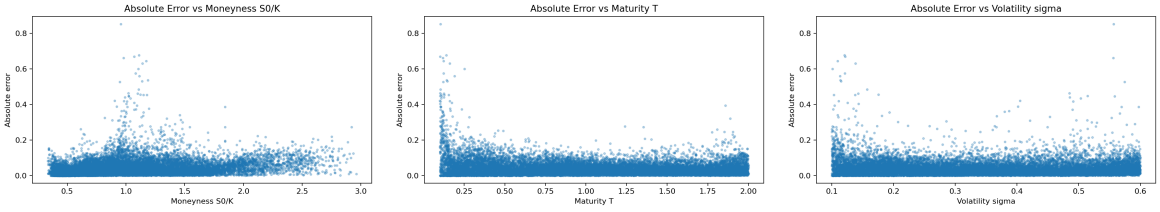


Figure 7.6: Distribution of neural network prediction errors.

The error distribution in Figure 7.6 is concentrated near zero, which is consistent with the low MAE and RMSE values reported in Table 7.1. This suggests that the final result is not only an artifact of aggregate averaging, but corresponds to generally small pointwise errors across the test set.



(a) Moneyness **(b)** Maturity **(c)** Volatility

Figure 7.7: Absolute neural network error against selected financial features.

Figure 7.7 checks whether errors are localized in specific regions of the parameter space. This diagnostic step is important because a pricing surrogate should not only have good global metrics, but should also behave reliably across financially meaningful regimes. The plots therefore complement the numerical metrics and provide a visual check of the approximation quality.

8 Discussion

8.1 Answer to the Research Question

The research question of the project is whether a feed-forward neural network can approximate the Black-Scholes pricing function for European call options. The experimental evidence supports a positive answer within the sampled parameter domain and under the assumptions of the Black-Scholes model.

In mathematical terms, the project studies the approximation problem

$$f_\theta(x) \approx C_{BS}(x), \qquad x = (S_0, K, T, r, \sigma), \tag{8.1}$$

or, in the final configuration,

$$f_\theta(S_0, K, T, r, \sigma, S_0/K) \approx C_{BS}(S_0, K, T, r, \sigma). \tag{8.2}$$

The final test-set results show very low absolute error and an R^2 close to one. This indicates that, on held-out synthetic contracts sampled from the same distribution as the training data, the trained neural network reproduces the analytical pricing map very accurately.

> *The strongest conclusion of the project is conditional but meaningful: within the Black-Scholes synthetic setting, the neural network is an accurate learned approximation of the analytical pricing function.*

This distinction is important. The project does not show that a neural network can price arbitrary real market options. It shows that, when the target pricing rule is known and deterministic, a supervised neural network can learn that rule with high accuracy and then evaluate it quickly.

8.2 What the Results Support

The results support three main claims. Table 8.1 summarizes each claim together with the corresponding evidence and the required caveat.

Claim	Evidence	Caveat
The neural network approximates Black-Scholes accurately	Final MAE, RMSE, R^2, true-vs-predicted plot, and error diagnostics	Valid on the sampled parameter domain, not automatically outside it
Neural inference is useful as a surrogate	Runtime benchmark shows much faster inference than Monte Carlo	Training has an upfront cost and the analytical formula is still faster when available
Controlled noise degrades performance predictably	Noisy-target experiments show increasing MAE and RMSE with increasing label noise	The noise is synthetic and should not be interpreted as real market microstructure

Table 8.1: Interpretation of the main empirical claims.

The first claim is the central one. The final model produces predictions that are close to the analytical Black-Scholes prices on the test set. This confirms that a relatively small MLP can approximate a smooth financial pricing function when the training data cover the relevant parameter ranges.

The second claim concerns computational role rather than mathematical exactness. The neural network is not proposed as a replacement for the closed-form Black-Scholes formula. When a closed-form formula exists, it remains exact and extremely fast. The more relevant comparison is with simulation-based pricing methods such as Monte Carlo, where repeated evaluation can be more expensive.

The third claim concerns robustness. Adding controlled label noise makes the learning problem harder, but the neural network does not collapse under moderate noise. This suggests that the model can partially smooth noisy supervision, although the experiment remains deliberately synthetic and controlled.

8.3 Role of Monte Carlo

Monte Carlo simulation plays two roles in the project. First, it is a classical quantitative benchmark for option pricing. Second, it provides a natural comparison for the surrogate-model idea.

For a European call option under Black-Scholes dynamics, Monte Carlo estimates

$$C_{BS} = e^{-rT}\mathbb{E}^{\mathbb{Q}}\left[(S_T - K)^+\right] \tag{8.3}$$

by the sample average

$$\hat{C}_{MC} = e^{-rT}\frac{1}{M}\sum_{j=1}^{M}(S_T^{(j)} - K)^+. \tag{8.4}$$

Its error decreases at the standard rate $O_P(M^{-1/2})$. This convergence rate is robust and model-independent, but it also explains why Monte Carlo can be computationally expensive when many paths or many contracts are required.

The neural network follows a different logic. It pays a training cost upfront, then produces prices through a deterministic forward pass:

$$\hat{C}_{NN} = f_\theta(x). \tag{8.5}$$

Therefore, the runtime comparison should be read as a comparison between repeated simulation and learned inference after training. It should not be read as evidence that the neural network is better than Black-Scholes itself, because the analytical formula remains the natural method when it is available.

8.4 Role of Feature Engineering and Activation Choice

The intermediate experiments help explain why the final configuration was selected. Adding moneyness exposes a financially meaningful ratio:

$$m = \frac{S_0}{K}. \tag{8.6}$$

This ratio does not add new information, since it is computed from S_0 and K. However, it gives the neural network direct access to a quantity that strongly affects the payoff regime of a call option. In practical terms, moneyness helps distinguish out-of-the-money, at-the-money, and in-the-money regions.

The activation-function experiment also has a natural interpretation. The Black-Scholes price is a smooth function of its inputs over the selected parameter ranges. Smooth activations such as Tanh and SiLU can therefore be well suited to this regression task. The final choice of SiLU is supported by the intermediate results and by the improved final run.

8.5 Error Sources and Diagnostic Plots

Even in a controlled synthetic setting, the neural network approximation is not exact. Errors can arise from several sources:

- finite training data;
- finite model capacity;
- imperfect optimization;
- numerical scaling and inverse-scaling effects;
- local difficulty in specific regions of the input space.

The diagnostic plots against moneyness, maturity, and volatility are therefore important. They check whether the aggregate metrics hide localized weaknesses. For example, if errors were systematically larger for short maturities or high volatilities, the global R^2 alone would not be sufficient to evaluate the model.

The project also clips negative neural-network predictions to zero after inverse scaling. This is financially reasonable because European call prices are non-negative:

$$C \geq 0. \tag{8.7}$$

However, this correction is still a post-processing step. It enforces a basic no-negativity constraint, but it does not guarantee all theoretical no-arbitrage properties of option prices, such

as monotonicity in S_0 or convexity in S_0.

8.6 Interpretation of Noisy Targets

The noisy-target experiments should be interpreted carefully. The noise model is:

$$\tilde{C} = \max(C_{BS} + \epsilon, 0), \qquad \epsilon \sim \mathcal{N}\left(0, [\alpha \max(C_{BS}, 1)]^2\right). \tag{8.8}$$

The training target is $\tilde{C}$, while evaluation remains against C_{BS}. This design answers the following question:

> *If the supervised labels are imperfect but still centered around the Black-Scholes price, can the model recover a good approximation of the clean pricing function?*

The experiments show that performance deteriorates as α increases. This is expected: noisy labels reduce the information available to the learner. At the same time, moderate noise does not destroy the approximation, which suggests some robustness of the neural-network surrogate.

This experiment should not be confused with real market modeling. Real option prices contain bid-ask spreads, liquidity effects, dividend assumptions, early-exercise considerations for some contracts, and implied-volatility structure. The controlled noisy-target experiment is therefore a robustness analysis, not a substitute for exchange-traded option data.

8.7 Limits of the Synthetic Setting

The use of synthetic data is appropriate for the research question because the target is the Black-Scholes pricing function itself. Using real exchange-traded option prices would change the problem. The model would no longer learn only the mathematical map $x \mapsto C_{BS}(x)$. Instead, it would learn a market-pricing mechanism affected by additional variables that are not present in the current input vector.

This is why the synthetic setting is not a weakness of the project. It is a methodological choice that makes the experiment controlled, reproducible, and directly measurable. The limitation is that the conclusions remain internal to the Black-Scholes world.

Aspect	Controlled synthetic setting	Real market setting
Target	analytical Black-Scholes price	observed market quote
Ground truth	known exactly	not directly known
Noise	controlled by design	market microstructure and liquidity
Evaluation	direct error against C_{BS}	depends on data quality and market assumptions
Research question	approximation of a known pricing map	modeling market prices

Table 8.2: Difference between the project setting and a real-market option-pricing setting.

8.8 Comparison with Classical Methods

The three main pricing approaches have different roles.

- Black-Scholes is the analytical reference and ground truth.
- Monte Carlo is a flexible numerical estimator and runtime benchmark.
- The neural network is a learned pricing surrogate after training.

SVR adds a fourth perspective: a classical machine-learning baseline on reduced datasets. It performs well, but its scalability is less favorable than the neural-network approach for larger training sets.

The final interpretation is therefore not a ranking that is valid in every situation. It is a role-based comparison:

$$\text{Black-Scholes} \rightarrow \text{ground truth}, \tag{8.9}$$

$$\text{Monte Carlo} \rightarrow \text{classical numerical benchmark}, \tag{8.10}$$

$$\text{NN} \rightarrow \text{fast learned surrogate}. \tag{8.11}$$

This role-based interpretation keeps the conclusion precise. The neural network is successful because it approximates the chosen pricing function accurately and evaluates quickly after training. It is not presented as universally superior to analytical or numerical methods.

9

Conclusions and Future Work

9.1 Final Answer to the Project Question

The project started from a precise research question:

> *Can a feed-forward neural network accurately approximate the Black-Scholes pricing function for European call options?*

The answer is positive within the controlled setting studied in the report. The final model learns the map

$$(S_0, K, T, r, \sigma, S_0/K) \mapsto C_{BS} \tag{9.1}$$

with high accuracy on held-out synthetic observations. Using 100,000 synthetic contracts, the final neural network achieves:

$$\text{MAE} = 0.04290, \qquad \text{RMSE} = 0.06208, \qquad R^2 = 0.999993. \tag{9.2}$$

These values show that the neural network reproduces the analytical Black-Scholes price very closely over the sampled parameter domain. The result is also supported visually by the true-vs-predicted plot and by the error diagnostics discussed in Chapter 7.

> *The project demonstrates that a supervised feed-forward neural network can act as an accurate pricing surrogate for the Black-Scholes pricing function in a controlled synthetic setting.*

This conclusion is intentionally limited to the scope of the experiment. The model is not claimed to price real exchange-traded options, and it is not presented as a replacement for the closed-form Black-Scholes formula. Instead, the contribution is to show how a neural network can learn a known pricing map and provide fast inference once trained.

9.2 Main Findings

The main findings can be summarized in Table 9.1.

Finding	Evidence	Interpretation
The neural network approximates Black-Scholes accurately	Low MAE and RMSE, R^2 close to one	The MLP learns the pricing map on the sampled domain
Moneyness and SiLU improve the final model	Intermediate experiments and improved final run	Financial feature engineering and smooth activations help regression
Neural inference is faster than Monte Carlo after training	Runtime benchmark on 1,000 pricing queries	The trained model is useful as a repeated-evaluation surrogate
Noisy labels reduce accuracy but do not destroy the approximation	Controlled noisy-target experiments	The model shows moderate robustness, within a synthetic noise setup
SVR is a useful classical reference	Reduced-scale SVR benchmark	Classical ML performs well, but is not the scalable main pipeline

Table 9.1: Summary of the main project findings.

The first finding is the core result of the project. It answers the research question directly. The remaining findings explain why the final model was selected, how it compares with classical methods, and how robust the approach is when the training labels are perturbed.

9.3 Role of the Neural Network

The neural network has a specific role in the project. It is not the source of the theoretical price. The theoretical price is still given by the Black-Scholes formula:

$$C_{BS} = S_0 N(d_1) - Ke^{-rT} N(d_2). \tag{9.3}$$

The neural network is instead a learned approximation:

$$\hat{C}_{NN} = f_\theta(x). \tag{9.4}$$

This distinction matters. When the analytical formula is available, Black-Scholes remains exact and faster. The neural network becomes interesting as a surrogate-model prototype: it shows how a learned model can approximate a pricing map and then produce repeated predictions through fast inference.

The Monte Carlo comparison clarifies this role. Monte Carlo remains a flexible numerical method, especially in settings where closed-form solutions are unavailable. However, it requires repeated sampling. The neural network requires training first, but after training it produces deterministic predictions without simulating paths.

9.4 Limitations

The main limitations of the project are not accidental weaknesses. They define the boundary of the research question.

- The dataset is synthetic and generated under Black-Scholes assumptions.
- Only European call options are considered.
- The target pricing function is known analytically.
- The input domain is bounded by the sampled parameter ranges.
- No-arbitrage constraints beyond non-negativity are not explicitly enforced.
- Noisy targets are controlled perturbations and should not be interpreted as real market prices.
- Real exchange-traded option data are not used, because that would change the research question from function approximation to market-price modeling.

The most important limitation is the synthetic setting. This project answers a clean mathematical question:

$$\text{Can a neural network approximate } C_{BS}? \tag{9.5}$$

It does not answer the different empirical-market question:

$$\text{Can a neural network explain observed option market prices?} \tag{9.6}$$

The second question would require additional variables, careful data cleaning, treatment of bid-ask spreads, dividends, liquidity effects, and implied-volatility structure. For this reason, keeping the current project synthetic is methodologically consistent.

9.5 Future Work

Several extensions are possible. They can be grouped according to how much they change the current research question.

Extension	Expected difficulty	Effect on the research question
Additional neural architectures	moderate	keeps the same Black-Scholes approximation task
More detailed error analysis by regime	low to moderate	strengthens the current evaluation
Greeks estimation	moderate	extends the surrogate from prices to sensitivities
Heston or stochastic-volatility models	high	changes the underlying pricing model
Basket or path-dependent options	high	increases dimensionality and computational relevance
American options	high	introduces early-exercise features and a different pricing problem
Real exchange-traded option data	high	changes the question to market-price modeling

Table 9.2: Possible future extensions and their impact on the project scope.

The most natural first extension would be a more detailed regime-based error analysis. For example, one could separately study deep out-of-the-money, at-the-money, and deep in-the-money contracts. This would keep the current research question unchanged while improving the understanding of where the neural surrogate is most reliable.

A second extension would be Greeks estimation. Since the neural network is differentiable, sensitivities could be estimated by automatic differentiation:

$$\Delta_{NN} = \frac{\partial f_\theta}{\partial S_0}, \qquad \mathcal{V}_{NN} = \frac{\partial f_\theta}{\partial \sigma}. \tag{9.7}$$

These quantities could then be compared with analytical Black-Scholes Greeks. This would be a natural continuation because option pricing is often used together with risk sensitivities.

More ambitious extensions include stochastic volatility, basket options, American options, and real market data. These are valuable directions, but they would move beyond the initial purpose of this project. They should therefore be treated as new research questions rather than small modifications of the current experiment.

9.6 Final Remarks

The project combines stochastic modeling, option pricing, Monte Carlo simulation, supervised learning, and reproducible software engineering. The final result is a complete experimental pipeline: synthetic data generation, analytical pricing, Monte Carlo benchmarking, neural-network training, SVR comparison, noisy-target robustness, runtime analysis, plots, tests, documentation, and a LaTeX report.

The main lesson is that neural networks can be useful in quantitative finance when their role is clearly defined. In this project, the neural network is not a black-box trading model. It is a controlled function approximator trained to reproduce a known pricing rule. This makes the experiment mathematically interpretable, computationally reproducible, and suitable as a foundation for more complex pricing problems.

A

Python Documentation

A.1 Purpose of the Documentation

The project includes a separate Python documentation site built with Sphinx. The purpose of this documentation is to make the implementation inspectable at module and API level, without overloading the main report with low-level interface details. The report and the Sphinx documentation therefore have complementary roles:

- the report explains the mathematical model, experimental design, software architecture, and empirical results;
- the Sphinx documentation exposes the package structure, module-level documentation, function signatures, class interfaces, and docstrings.

This separation keeps the report focused on the scientific content while still making the codebase auditable.

A.2 Sphinx Documentation Structure

The documentation sources are stored in:

```
docs/source/
```

Listing A.1: Source directory for the Sphinx documentation.

The API reference is generated from the docstrings contained in the Python package:

```
src/nn_option_pricing/
```

Listing A.2: Main Python package documented through Sphinx autodoc.

The most important documentation entry points are summarized in Table A.1.

File or directory	Role
`docs/source/index.rst`	main documentation index
`docs/source/overview.rst`	project orientation and repository overview
`docs/source/methodology.rst`	methodological description of the experiment
`docs/source/experiment_pipeline.rst`	end-to-end experiment pipeline
`docs/source/experiments.rst`	additional experiment descriptions
`docs/source/testing.rst`	testing and verification strategy
`docs/source/api/`	generated API reference for the Python package

Table A.1: Main Sphinx documentation entry points.

A.3 Local Documentation Build

The documentation can be built locally from the repository root after installing the project and documentation dependencies:

```
pip install -r requirements.txt
pip install -r requirements-docs.txt
pip install -e . --no-build-isolation
```

Listing A.3: Installation commands for local documentation builds.

The standard HTML build command is:

```
sphinx-build -b html docs/source docs/build/html
```

Listing A.4: Local Sphinx HTML build command.

The generated documentation can then be opened from:

```
docs/build/html/index.html
```

Listing A.5: Local HTML documentation entry point.

A.4 Strict Documentation Build

During development, it is useful to treat documentation warnings as errors. This makes missing references, malformed directives, or broken API documentation visible immediately:

```
sphinx-build -W -b html docs/source docs/build/html
```

Listing A.6: Strict Sphinx build command.

The same strict build is used in the automatic documentation workflow described below.

A.5 Automatic Deployment with GitHub Actions and GitHub Pages

The generated HTML documentation is not tracked in the repository. This is intentional: `docs/build/html/` is a generated artifact, and keeping it out of version control avoids unnecessary file churn. Instead, the documentation can be built automatically through GitHub Actions and published with GitHub Pages.

The workflow is stored in:

```
.github/workflows/docs.yml
```

Listing A.7: GitHub Actions workflow for building and publishing the documentation.

At each push to the main branch, the workflow performs the following operations:

1. checks out the repository;
2. installs Python and the project dependencies;
3. installs the documentation dependencies;
4. builds the Sphinx documentation using the strict `-W` option;
5. uploads the generated HTML site as a GitHub Pages artifact;
6. deploys the artifact to GitHub Pages.

This approach keeps the repository clean while still providing an online version of the documentation. Once GitHub Pages is enabled for the repository, the documentation is available through the repository Pages URL.

B

Code Navigation

B.1 How to Read the Codebase

The implementation is organized as a reusable Python package plus a small set of command-line scripts. For a reader who wants to inspect the code without reading every file line by line, the most useful path is:

1. start from `scripts/run_experiment.py`, which defines the command-line interface for the main experiment;
2. inspect `src/nn_option_pricing/config.py`, where experiment parameters are represented as immutable configuration objects;
3. follow `src/nn_option_pricing/pipeline.py`, which connects the main experimental stages;
4. inspect the specialized modules only when needed.

This gives a top-down view of the project before moving into the individual numerical and machine-learning components.

B.2 Main Code Entry Points

Table B.1 summarizes the most important files for understanding the implementation. Except for `run_experiment.py`, which is in `scripts/`, the listed modules are located in `src/nn_option_pricing/`.

File	Role
`run_experiment.py`	main experiment command-line interface
`config.py`	experiment configuration dataclasses and output paths
`pipeline.py`	end-to-end clean-target neural-network experiment
`black_scholes.py`	analytical Black-Scholes pricing and payoff functions
`dataset.py`	synthetic dataset generation and feature construction
`training.py`	neural-network training, scaling, prediction, and early stopping
`monte_carlo.py`	batched Monte Carlo benchmark
`evaluation.py`	regression metrics used in the experiments
`plots.py`	diagnostic plots saved by the pipeline
`svr.py`	reduced-scale Support Vector Regression benchmark
`noise.py`	controlled target-noise generation

Table B.1: Suggested code entry points for project inspection.

B.3 Experiment Scripts

The repository contains separate scripts for the main experiment and for additional benchmarks. This keeps the main pipeline focused while making each extension reproducible.

```
scripts/
    run_experiment.py               # main neural-network experiment
    benchmark_runtime.py            # pricing runtime comparison
    run_svr_benchmark.py            # clean-target SVR benchmark
    run_noisy_targets_experiment.py # noisy-target neural-network experiment
    run_noisy_svr_benchmark.py      # noisy-target SVR benchmark
```

Listing B.1: Main experiment and benchmark scripts.

B.4 GitHub Code Links

When the report is read together with the GitHub repository, code links can be used to inspect specific implementation details. For stable references, it is preferable to link to a fixed commit rather than to a moving branch. This avoids broken or misleading line references if the code changes later.

The most useful targets for direct GitHub links are:

- the analytical pricing implementation in `black_scholes.py`;
- the synthetic dataset generator in `dataset.py`;
- the training loop in `training.py`;
- the end-to-end pipeline in `pipeline.py`;
- the Monte Carlo benchmark in `monte_carlo.py`;
- the SVR and noisy-target extensions in `svr.py`, `noise.py`, and the noisy experiment modules.

These links are not a substitute for the report, but they make it easier for a reader to verify how the mathematical and experimental choices are implemented in code.

Bibliography

[1] Fischer Black and Myron Scholes. The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3):637–654, 1973.

[2] George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2:303–314, 1989.

[3] Paul Glasserman. *Monte Carlo Methods in Financial Engineering*, volume 53 of *Stochastic Modelling and Applied Probability*. Springer New York, 2004.

[4] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989.

[5] Andrea Pascucci and Wolfgang J. Runggaldier. *Financial Mathematics: Theory and Problems for Multi-period Models*. UNITEXT. Springer Milan, 2012.

[6] Steven E. Shreve. *Stochastic Calculus for Finance I: The Binomial Asset Pricing Model*. Springer Finance. Springer New York, 2004.

[7] Steven E. Shreve. *Stochastic Calculus for Finance II: Continuous-Time Models*. Springer Finance. Springer New York, 2004.

[8] Alex J. Smola and Bernhard Schölkopf. A tutorial on support vector regression. *Statistics and Computing*, 14(3):199–222, 2004.